SQL Injection Lab: Detect, Exploit, Fix

UneeQ Interns — Cybersecurity Track

1. Scope & Environment

- **Target application:** DVWA (Damn Vulnerable Web Application), bundled on the Metasploitable2 VM
- **Environment:** Metasploitable2 running as an isolated VirtualBox guest, Host-only network adapter (not exposed to the internet or LAN)
- **Access:** `http://192.168.56.101/dvwa/`
- **Authorization:** Self-hosted, deliberately vulnerable lab application intended for security training. No real or production systems were accessed.
- **Module tested:** SQL Injection, across all three built-in DVWA security levels (Low, Medium, High)

2. Vulnerability Found

The `SQL Injection` module accepts a `User ID` parameter (`id`) and passes it into a SQL query used to look up user records. Depending on the configured security level, this parameter is handled with varying degrees of sanitization, exposing different classes of SQL injection.

2.1 Low Security — Full Injection

Source code:

```
$id = $_GET['id'];
$getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
$result = mysql_query($getid) or die('<pre>' . mysql_error() . '</pre>');
```

The `id` parameter is concatenated directly into the query string inside single quotes, with **no sanitization or type checking** at all.

Payload 1 — authentication bypass pattern:

```
1' OR '1'='1
```

Result: returned multiple user records instead of a single user, confirming the WHERE clause was successfully manipulated.

Payload 2 — data exfiltration via UNION:

```
1' UNION SELECT user, password FROM users-- -
```

Result: dumped all usernames and password hashes from the `users` table, including `admin`, `gordonb`, `1337`, `pablo`, and `smithy`, each with their corresponding MD5 hash.

Root cause: Unsanitized, unescaped, directly concatenated user input inside a SQL string. Any attacker-controlled quote character can break out of the intended query structure.

2.2 Medium Security — Partial Mitigation, Bypassed

Source code:

```
$id = $_GET['id'];
$id = mysql_real_escape_string($id);
$getid = "SELECT first_name, last_name FROM users WHERE user_id = $id";
$result = mysql_query($getid) or die('<pre>' . mysql_error() . '</pre>');
```

Medium applies `mysql_real_escape_string()`, which escapes single quote characters. Notably, the query itself **no longer wraps `$id` in quotes** (`user_id = $id`, not `user_id = '$id'`).

Test 1 — quote-based payload (same as Low):

```
1' OR '1'='1
```

Result: **blocked**. The escaped quote broke the query syntax, producing a MySQL syntax error (`...near '\' OR \'1\'=\'1\'...`).

Test 2 — quote-less UNION payload:

```
1 UNION SELECT user, password FROM users-- -
```

Result: **successful**. Because the query has no surrounding quotes and no numeric type validation, a UNION-based payload that doesn't rely on quote characters bypasses the escaping entirely and dumps the full `users` table (usernames + password hashes) exactly as at Low security.

Finding: Medium's mitigation is incomplete. It defends against the specific attack pattern it was designed for (quote-breakout) but does not validate that `id` is actually numeric, leaving the endpoint fully exploitable via an alternate injection technique.

2.3 High Security — Effectively Mitigated

Source code:

```
$id = $_GET['id'];
$id = stripslashes($id);
$id = mysql_real_escape_string($id);

if (is_numeric($id)) {
    $getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
    $result = mysql_query($getid) or die('<pre>' . mysql_error() . '</pre>');
    ...
}
```

High adds `stripslashes()`, `mysql_real_escape_string()`, **and** a critical `is_numeric($id)` check before the query ever runs. Both the quote-based (Low) and quote-less UNION (Medium-bypass) payloads were tested here.

Result: both payloads were rejected, since any non-numeric input fails the `is_numeric()` check before reaching the database. This is the first level that validates the *type* of input rather than only attempting to neutralize special characters.

3. Comparison Summary

Level	Escaping	Type validation	Quote-based injection	Quote-less UNION injection
Low	None	None	✅ Successful	✅ Successful
Medium	`mysql_real_escape_string()`	None	❌ Blocked	✅ Successful (bypass)
High	`stripslashes()` + `mysql_real_escape_string()`	`is_numeric()`	❌ Blocked	❌ Blocked

4. The Fix

The underlying issue at every vulnerable level is that user input is concatenated into a SQL string rather than passed as a bound parameter. Escaping functions and type checks are workarounds; the correct fix is a **parameterized (prepared) statement**, which separates SQL code from data entirely so injected syntax can never be interpreted as part of the query.

Vulnerable pattern (Low/Medium):

```
$id = $_GET['id'];
$getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
$result = mysql_query($getid);
```

Fixed version using MySQLi prepared statements:

```
$id = $_GET['id'];

$stmt = $mysqli->prepare("SELECT first_name, last_name FROM users WHERE user_id = ?");
$stmt->bind_param("s", $id);
$stmt->execute();

$result = $stmt->get_result();

while ($row = $result->fetch_assoc()) {
    echo '<pre>';
    echo 'ID: ' . htmlspecialchars($id) . '<br>First name: ' .
htmlspecialchars($row['first_name']) .
         '<br>Surname: ' . htmlspecialchars($row['last_name']);
    echo '</pre>';
}

$stmt->close();
```

Why this works: the `?` placeholder is bound as data via `bind_param()`, never concatenated into the query string. Even a malicious payload like `1' OR '1'='1` or `1 UNION SELECT user, password FROM users-- -` is treated as a literal string value being searched for, not as executable SQL — so both the Low and the Medium-bypass payloads fail against this version, with no dependence on escaping or numeric-only filtering. `htmlspecialchars()` is also added on output to mitigate stored/reflected XSS as a secondary hardening step, independent of the SQLi fix.

5. Retest Confirmation

Re-running both previously successful payloads (`1' OR '1'='1` and `1 UNION SELECT user, password FROM users-- -`) against the parameterized version above returns no matching rows and no SQL errors — the input is treated purely as a data value for the `user_id` lookup, confirming the fix closes the vulnerability found at both Low and Medium levels.

6. Lessons Learned

- Escaping and blocklist-style filtering (as used at DVWA's Medium level) can give a false sense of security: they close the specific attack pattern they were designed for, but input validation gaps (here, missing numeric type checking) leave the application exploitable through a different technique.
- Defense needs to validate the **shape/type** of expected input (e.g. `is_numeric()`), not just neutralize special characters.
- Parameterized queries remove the entire class of SQL injection by design, rather than relying on correctly anticipating every possible malicious input pattern.
- Testing a single payload against a single security level is not sufficient to claim an application is "fixed" — the same target should be retested with varied payload techniques (quote-based vs. quote-less, UNION vs. boolean-based, etc.) at every mitigation level.